Enterprise WhatsApp platform technology stack

Companion to Architecture v2 | 30 September 2026

Recommended baseline: AWS + Go + React/TypeScript + PostgreSQL + Kafka + Temporal. This is a proposed stack for your product, with large-scale enterprise operation required from launch. It is not a claim about WhatChimp's private implementation.

Use managed infrastructure where possible. Keep messaging policy, tenant permissions, workflow definitions and dispatch decisions under your application's control. The selections below make the architecture concrete; exact versions, instance sizes and regions remain engineering validation decisions.

1. Application stack

Component	Selection	Responsibility
Web application	React + TypeScript + Next.js	Shared inbox, campaigns, reporting and administration.
UI components	Tailwind CSS + Radix UI	Consistent styling and accessible interaction primitives.
Automation editor	React Flow	Edit typed graphs; publish validated, immutable JSON workflow definitions.
Browser server state	TanStack Query	API fetching, cache invalidation and refresh.
Backend services	Go	Business APIs, ingress, dispatch, imports and integrations.
API contracts	REST + OpenAPI; gRPC + Protobuf	External REST contracts; typed internal service calls.
Database access	pgx + sqlc	Bounded connection pools and typed SQL access.
Realtime delivery	Dedicated Go WebSocket gateways	Agent updates, assignments and presence, with durable catch-up.
Mobile agent app	React Native + TypeScript	Mobile inbox and device registration for APNs/FCM.
Provider integration	Meta WhatsApp Cloud API adapter	Isolate payloads, error classes, capabilities and API versions.

Application boundary

Business rules live in Go services. Next.js serves the web application. Never execute arbitrary code supplied by the visual builder. A server compiler validates graph structure, permissions, node configuration and limits before publication.

Multiple Next.js instances require coordinated caching and deployment-version handling. Tenant-sensitive responses must not enter shared caches without a correct isolation policy. [1]

2. Data and processing stack

Component	Selection	Responsibility
System of record	Amazon Aurora PostgreSQL	Per-cell transactional state: tenants, contacts, conversations, message intents, consent, billing records and outbox.
Connection control	Bounded pgx pools; RDS Proxy where validated	Cap aggregate database connections. Validate proxy behavior with tenant context, transaction patterns and failover. [2]
Cache and presence	Amazon ElastiCache for Valkey	Rebuildable caches, presence and short-lived coordination.
Event backbone	Amazon MSK Provisioned / Apache Kafka	Durable ingress, domain events, replay and independently scaled consumers. [3]
Event contracts	Protobuf + CI compatibility checks	Versioned envelopes and schemas; reject incompatible producer/consumer changes.
Workflow engine	Temporal Cloud, subject to residency	Bot execution, campaign waits, SLA timers and integration retry orchestration. [4]
Object storage	Amazon S3 + KMS	Attachments, imports, exports, knowledge files and audit archives.
Search	Amazon OpenSearch Service	Tenant-filtered search projections for contacts and conversations. [7]
Analytics	Amazon Redshift + S3	Campaign reporting and historical aggregation, isolated from operational SQL.

Clear ownership prevents conflicting truth

PostgreSQL owns committed business state. Kafka transports durable records and events. Temporal owns durable workflow progress and timers. Valkey holds information that can be rebuilt. Search and analytics are derived projections and may lag.

Write business changes and an outbox row in the same SQL transaction. A relay publishes committed rows to Kafka. Consumers deduplicate and atomically apply effects where possible. A Kafka acknowledgment alone does not mean the business action completed.

Use transactional records for strict shared quota reservations and dispatch ownership. A cache loss must not reset spend or sending allowances. Partition ingress independently from conversation events, then explicitly normalize and re-key as defined in Architecture v2.

Managed-service boundary

Select Temporal Cloud only after confirming acceptable regions, payload handling, connectivity and recovery behavior. If these requirements cannot be met, deploy Temporal inside the approved environment. That is a deployment decision, not a change to workflow semantics.

3. Infrastructure and operations

Layer	Selection	Implementation intent
DNS and delivery	Route 53 + CloudFront	Resolve regional routes and deliver frontend assets.
Edge protection	AWS WAF	Request filtering and abuse controls.
Regional routing	Application Load Balancers	Route HTTPS APIs, webhooks and WebSocket connections.
Runtime	Amazon EKS	Private worker nodes across three availability zones; separate pools and budgets by workload.
Egress	Controlled proxy + NAT gateways	Approved external destinations; zone-local routes where applicable.
Private service access	VPC endpoints / PrivateLink	Use private access where the selected service supports it.
Secrets and encryption	Secrets Manager + KMS	Credential rotation, encryption and scoped workload access.
Customer identity	Amazon Cognito	Sign-in and enterprise SAML/OIDC federation. [5]
Authorization	Backend permission model	Tenant, workspace and delegated-agency permissions on every operation.
Infrastructure as code	Terraform	Reviewed definitions for accounts, networks, compute, storage and policies.
Delivery	GitHub Actions + ECR + Argo CD + Helm	Build, scan, sign, promote and reconcile versioned deployments.
Telemetry	OpenTelemetry + CloudWatch + Managed Prometheus + Managed Grafana	Correlated logs, metrics, traces and service-level dashboards.
Verification	Go tests + Testcontainers + Playwright + k6	Behavior, integration, browser and capacity checks; complement with recovery exercises.

Operational guardrails

Separate production, nonproduction and security accounts. Use temporary workload credentials, explicit network policies and private data endpoints. Regional deployments and their recovery environments must be independently operable. [6]

Cognito authenticates users; it does not implement the product's agency hierarchy or row-level tenant access. Resolve authenticated identity to explicit workspace grants. Never trust a tenant identifier solely because the client supplied it.

Monitor Kafka lag, ingress acknowledgment latency, dispatch ambiguity, provider errors, workflow queue age, database saturation and WebSocket catch-up failures. Route actionable alerts to named service owners.

4. How the technologies connect

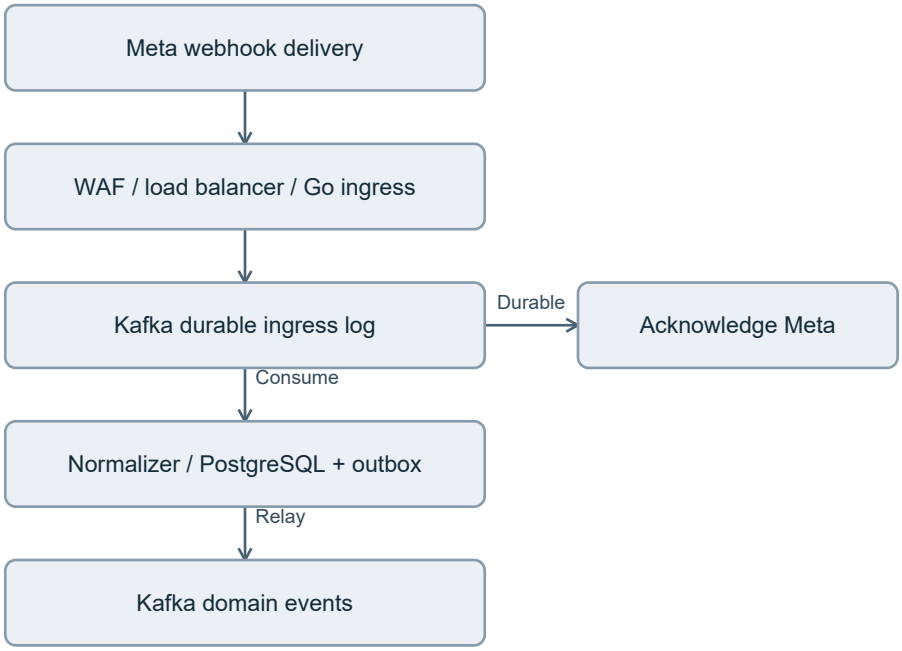


Figure 1. Inbound acceptance precedes asynchronous processing. Cross-region routing acknowledges only after the required durable receipts have returned.

Event consumer	Result
Temporal workers	Advance the appropriate workflow using idempotent processing.
Go realtime gateway	Notify agents; recover missed updates from durable state.
Search indexer	Update the tenant-filtered OpenSearch projection.
Analytics consumer	Load S3 / Redshift reporting data with deduplication.

Outbound path

Agent, campaign or workflow → Go command API → PostgreSQL intent and outbox → Kafka → policy and quota scheduler → Go dispatch gateway → egress proxy → Meta.

Before transmission, check consent, template/window eligibility, account health, scoped quotas, spend, expiry and current ownership. Persist the dispatch attempt before calling the provider. A timeout after possible transmission enters UNKNOWN; it does not trigger a blind resend.

Independent delivery-status path

Meta → webhook ingress → durable event pipeline → status reconciler → PostgreSQL → inbox and analytics. Meta originates the callback independently of your outbound adapter. Correlate provider IDs or supported reference fields and tolerate receipts arriving before the HTTP response is stored.

Public links use HTTPS. Internal service calls and data connections use TLS with scoped identities; use mTLS where configured. A browser never receives direct database, Kafka or provider credentials.

5. Enterprise deployment decisions

Regional cells are the unit of capacity and isolation

Deploy application workers, transactional storage, event processing and controlled sending capacity with explicit cell ownership. Spread a production cell across three availability zones. Provision dedicated cells when customer isolation requirements justify them. Independent regions host independent clusters; a Kubernetes cluster does not span regions.

Separate worker pools and resource budgets for ingress/dispatch, workflow execution, imports and analytics. A large import or report must not consume the capacity reserved for accepting messages or replying to customers.

Recovery exists from launch

Deploy recovery infrastructure and run restoration and failover exercises before general availability. Coordinate database promotion, Kafka recovery, workflow failover, routing and tenant ownership epochs. There must be one active sending owner for each tenant.

Cross-region replication can lag. Define and test recovery-point and recovery-time objectives for each store. Managed replication does not provide end-to-end exactly-once external delivery. Temporal failover must be coordinated with business-data and sender ownership. [3, 4]

Decision to pin	Evidence required before release
Regions and residency	Customer geography, processing and storage rules, vendor-region support and recovery-location approval.
Service and language versions	Supported releases, compatibility matrix, upgrade rehearsals and dependency policy.
Cell size and database layout	Peak workloads, tenant skew, query plans, connection limits, storage growth and failover results.
Kafka partitions and retention	Ingress and event rates, replay horizon, consumer parallelism, ordering scope and hot-key behavior.
Temporal topology	Workflow histories, timer volumes, task latency, encryption and regional recovery needs.
Search and analytics sizing	Index growth, query concurrency, retention, projection lag and access-control tests.
Identity lifecycle	Enterprise SSO onboarding, session policy, deprovisioning and any required SCIM integration.
Provider contracts	Pinned Meta API version, required scopes, webhook subscriptions, fixture tests and current quota model.

Release evidence

Demonstrate tenant isolation, duplicate-webhook handling, unknown-outcome safety, bounded retry, durable inbox catch-up, overload protection and regional recovery. The tools enable these controls; the application and operating procedures must prove them.

The architecture remains enterprise-scale from day one. Exact cluster counts and instance sizes are intentionally not invented without workload and residency evidence.

6. References and implementation scope

The recommendations consolidate the technology-stack discussion and align with Enterprise WhatsApp Platform Architecture v2. The official references below support selected platform capabilities; they do not establish WhatChimp's internal implementation or guarantee this system's capacity.

[1] Next.js: Self-hosting

Multi-instance caching, deployment coordination and hosting considerations.

Open official reference

[2] AWS: RDS Proxy concepts

Managed connection pooling; validate engine and application-specific behavior.

Open official reference

[3] AWS: Amazon MSK features

Managed Kafka and cross-region replication capabilities.

Open official reference

[4] Temporal: High-availability failovers

Official documentation source for namespace failover and replication behavior.

Open official reference

[5] AWS: Cognito authentication

Authentication and enterprise identity-provider federation.

Open official reference

[6] AWS: EKS multi-account strategy

Workload account separation and EKS deployment boundaries.

Open official reference

[7] AWS: OpenSearch multi-AZ deployment

Managed search deployment and availability-zone configuration.

Open official reference

Scope and status

This document selects technologies and describes their roles. It does not constitute deployed infrastructure, executed load tests, a cost quote or a production certification. Pin exact versions and validate current provider contracts during implementation.

AI model selection, commerce/payment integrations and detailed billing-provider selection remain separate decisions from this core stack. If AI assistance is enabled, preserve the architecture's tenant-scoped retrieval, controlled model gateway, audit trail and human handoff requirements.